Manual técnico

Robot de laberinto · Microcontroladores 2

Cómo leerlo. Cada sección empieza con un recuadro **En simple**, que se entiende sin ser ingeniero. Lo que sigue es el detalle técnico. Al final hay un glosario.

1. Qué hace el robot

En simple: el robot entra a un laberinto que no conoce, busca el centro, regresa a la salida y cuenta los cuadros de colores del piso. Mientras avanza dibuja un mapa. En el siguiente intento usa ese mapa para ir directo y más rápido.

- **Laberinto:** 16 × 16 celdas de 180 mm; la meta son las 2 × 2 celdas centrales.
- **Robot:** diferencial con dos ruedas motrices y un patín, huella ≤ 120 mm.
- **Estrategia en dos fases:**
    1. **Exploración** con *flood fill*: origen → meta → origen.
    2. **Corrida rápida** sobre el mapa guardado, solo por celdas ya visitadas.

2. Dos cerebros

En simple: el Arduino UNO Q trae dos computadoras en una placa.

- Una **pequeña y muy puntual**, que mueve los motores y lee los sensores cien veces por segundo.
- Una **grande con Linux**, que piensa el camino, guarda el mapa y procesa la cámara.

Se pasan mensajes entre ellas. Si la grande tarda en contestar, la pequeña termina el movimiento y frena, así que el robot nunca queda fuera de control.

Dominio	Procesador	Sistema	Tareas
Tiempo real	STM32U585 (Cortex-M33)	Zephyr (sketch Arduino)	Control de motores, sensores, clasificación de color, pantalla y botón
Planeación	Qualcomm QRB2210 (Cortex-A53)	Debian Linux	Mapa, *flood fill*, trayectoria, persistencia, visión (OpenCV)

Enlace: RPC MessagePack sobre el enlace interno, enrutado por el servicio `arduino-router`. Linux invoca procedimientos que expone el MCU:

Procedimiento	Qué hace
`mover(n, rápido)`	Avanza *n* celdas. No bloquea; devuelve 0 si ya hay un movimiento en curso
`girar(k)`	Gira en sitio *k* × 90°; el signo indica el sentido
`ocupado()`	Devuelve 1 mientras hay un movimiento activo
`paredes()`	Máscara de 3 bits: izquierda, frente, derecha
`color()`	Último color visto, codificado como `c + 16·k` (clase *c*, a *k* celdas del inicio del movimiento)
`arranque()`	Evento del botón: 1 = corto, 2 = largo
`set_cuenta`, `set_estado`	Actualizan la pantalla

Separar los dominios aísla la latencia no determinista de Linux (planificador, E/S, visión) del lazo de control.

Respaldo: si el UNO Q falla, el mismo chasis acepta un UNO R3 (ATmega328P) con firmware *bare-metal* que hace todo en un solo chip:

- Sin Arduino core.
- Driver de I²C (TWI) propio.
- Base de tiempo con Timer1 en modo CTC.
- PWM rápido de 8 bits por registros en Timer0.
- El mismo *flood fill*.

3. Sensores de distancia

En simple: tres sensores láser (al frente y a los lados) miden en milímetros qué tan lejos está la pared, unas 50 veces por segundo. Si una pared está a menos de 12 cm, el robot la considera “pared”.

- **Sensores:** 3 × VL53L0X, tiempo de vuelo a 940 nm.
- **Direcciones I²C:** los tres traen la misma (0x29), que además comparte el sensor de color. Al arrancar se mantienen apagados con XSHUT (A0–A2). Se encienden uno por uno y cada uno se reasigna a 0x30–0x32 antes de `init()`.
- **Lectura:** modo continuo con *timing budget* de 20 ms (≈ 50 Hz por sensor). Es no bloqueante: solo se lee el sensor cuyo `RESULT_INTERRUPT_STATUS` indica dato nuevo.
- **Filtros:** lecturas de 0 mm o de más de 2000 mm cuentan como fuera de rango. Hay pared si $d < 120$ mm.

4. Sensor de color

En simple: debajo del robot hay un sensor con su propia lucecita, tapado con una capucha para que no le afecte la luz del lugar. Mide cuánto rojo, verde y azul refleja el piso y lo compara con los colores que le enseñamos en la calibración. Para no contar dos veces el mismo cuadro, exige ver el mismo color tres veces seguidas y no vuelve a contar hasta regresar al piso normal.

Adquisición

- TCS34725 con integración de 24 ms, ganancia 4× y LED propio, dentro de una capucha impresa.
- La capucha fija la distancia al piso y bloquea la luz ambiente.

Clasificación por vecino más cercano. Primero se normaliza la cromaticidad, que da invariancia de primer orden a la intensidad de luz:

> $\hat{r} = R / (R + G + B)$, igual para $\hat{g}$ y $\hat{b}$

La distancia a cada color de referencia i suma un término logarítmico de brillo. Ese término separa colores de tono parecido y distinta reflectancia, como blanco y amarillo:

> $d_i = (\hat{r} - r_i)^2 + (\hat{g} - g_i)^2 + (\hat{b} - b_i)^2 + 0.01 \cdot \ln^2((C + 1) / (C_i + 1))$

Si la menor distancia es mayor que 0.02, el resultado es “piso”.

Antirrebote: se exigen 3 muestras iguales seguidas (3 × 30 ms). El evento solo se dispara en la transición piso → color.

¿En qué celda estaba? Durante un avance de duración T por celda, la marca está en la celda $k = \lfloor t/T + \delta + 0.5 \rfloor$. Aquí $\delta = 0.2$ celdas es la distancia del eje de las ruedas al sensor. Python descarta las marcas repetidas por coordenada.

5. Cámara (opcional)

> **En simple:** la cámara en el mástil mira una celda adelante. Si cree ver una pared, el mapa la anota como “posible”, nunca como segura. Solo cuentan como seguras las que confirman los sensores láser. Si la cámara se equivoca y el robot se queda sin salida, el robot la ignora. Sin cámara, el robot funciona igual.

- **Captura:** cámara UVC a 320 × 240, OpenCV con V4L2.
- **Detección:** segmentación HSV del color de pared y fracción de píxeles en 3 regiones de interés de la celda siguiente (fondo, izquierda, derecha).
- **Mapa tentativo:** lo detectado va a un mapa separado del confirmado. El *flood fill* lo usa para anticipar callejones sin salida. Si deja al robot sin camino, se recalcula sin él.

6. Cómo se mueve

En simple: el robot no tiene sensores en las ruedas, así que mide las distancias por tiempo: “avanzar una celda” es avanzar unos 600 ms. Para no acumular error usa tres trucos:

1. **Arranca y frena suave** para que las llantas no patinen.
2. **Se mantiene centrado** midiendo las paredes de los lados.
3. **Si hay pared enfrente, frena por distancia** y no por tiempo, así corrige lo que se haya desviado.

Actuación

- Dos micromotorreductores DC (≈ 12 × 11 mm de sección), ruedas de 45 mm, puente H L298N.
- PWM en ENA/ENB y sentido en IN1–IN4.
- Freno activo: ambas entradas en alto con EN = 255 durante 60 ms.

Lazo de control a 100 Hz ($T = 10$ ms), en lazo abierto de posición:

1. **Perfil trapezoidal.** Rampa de +6 PWM por ciclo desde `VEL_MIN`, crucero y desaceleración lineal en el último cuarto de la duración nominal.
2. **Centrado proporcional.** $u = K_c \cdot e$, saturado en ±40 PWM y aplicado en diferencial: $PWM_izq = v - u$, $PWM_der = v + u$. El error e depende de las paredes:
   - Dos paredes: $e = d_izq - d_der$.
   - Una pared: error contra la distancia nominal `D_LADO`.
   - Ninguna: compensación estática `AJUSTE_DER`.
3. **Paro por referencia externa.** Con pared frontal, el avance termina cuando $d_frente \le$ `D_FRENTE`, con una tolerancia de $+T/3$. Así el error longitudinal se reinicia en cada celda con pared al frente.
4. **Giros en sitio** por tiempo (`MS_90`) a PWM constante.

Limitación reconocida: sin encoders ni IMU, los avances sin pared al frente y los giros dependen del voltaje de la batería y de la fricción.

- **Cómo lo mitigamos:** recalibración en sitio (`MS_CELDA`, `MS_90`), paro por pared y centrado continuo.
- **Mejora natural:** encoders en cuadratura y un giroscopio para cerrar el lazo en posición y rumbo.

7. Cómo encuentra el camino

> **En simple:** imagina que derramas agua en la meta. El agua avanza celda por celda y no cruza paredes. Cada celda guarda cuántos pasos tardó en llegarle el agua. El robot siempre se mueve a la celda vecina con el número más bajo, hasta llegar a la meta. Cada vez que descubre una pared nueva, vuelve a “derramar el agua”.

Mapa

- 16 × 16 celdas con 4 bits por celda (N, E, S, O).
- Cada pared se escribe también en la celda vecina.
- El perímetro empieza cerrado y una matriz aparte marca las celdas visitadas.

***Flood fill*.** BFS desde el conjunto objetivo, O(N^2) = 256 celdas, recalculado tras cada celda.

Exploración (supuesto optimista: lo desconocido es libre)

- Ante empate, prefiere seguir de frente, luego derecha, izquierda y media vuelta. Así minimiza giros, que son la mayor fuente de error.
- Recorre origen → meta → origen. El regreso explora otras rutas y aumenta la probabilidad de que el camino óptimo quede en celdas visitadas.

Corrida rápida (supuesto pesimista)

- Solo cruza aristas entre dos celdas visitadas.
- Junta los tramos rectos en un solo `mover(n)`, así no frena en cada celda.

Persistencia: el mapa y las marcas se guardan en JSON en la placa y sobreviven al apagado. Un toque largo del botón los borra.

8. Hardware

En simple: una batería de 7.4 V alimenta los motores y la placa. Todos los sensores comparten dos cables de datos (I²C). El chasis lo diseñamos nosotros en 3D, con medidas exactas, y lo imprimimos. Comprobamos por computadora que ninguna pieza choque con otra antes de imprimir.

Subsistema	Implementación
Alimentación	Batería 2S (6.0–8.4 V) → interruptor → L298N (V_S) y VIN del UNO Q (7–24 V, reguladores conmutados)
Lógica de potencia	Regulador 78M05 del L298N (necesita ≳ 7 V). Entradas compatibles con 3.3 V (V_IH ≥ 2.3 V)
Bus I²C	400 kHz, alimentado a 3.3 V para no exponer los pines del Q a *pull-ups* de 5 V. Lleva 3 × VL53L0X, TCS34725 y una OLED SSD1306
Pantalla	OLED 128 × 64 con driver propio. Muestra estado, conteos y un contador de *heartbeat* de la comunicación entre los procesadores
Chasis	OpenSCAD paramétrico de dos pisos, FDM (Bambu X2D), insertos de latón M3 y mástil de aluminio 2020. Interferencias verificadas por intersección CSG
Cambio de placa	Mismo mapa de pines en Q y R3 vía Proto Shield; solo cambia el puente de VCC (3V3 ↔ 5V)

Voltaje en los motores: el L298N es bipolar (Darlington) y cae ≈ 2–3 V. Con 7.4 V llegan ≈ 4.5–5 V a los motores. Es la principal ineficiencia del diseño; un puente MOSFET (TB6612FNG, DRV8833) la eliminaría.

9. Parámetros calibrables

Parámetro	Base	Qué controla
`MS_CELDA` / `MS_RAPIDA`	600 / 400 ms	Tiempo por celda (explorar / rápido)
`MS_90`	250 ms	Tiempo de giro de 90°
`VEL_EXPLORA` / `VEL_RAPIDA` / `VEL_GIRO` / `VEL_MIN`	140 / 220 / 130 / 80	PWM de 0 a 255
`KC`	0.8 PWM/mm	Fuerza del centrado
`PARED`	120 mm	Distancia para considerar pared
`D_FRENTE` / `D_LADO`	55 / 50 mm	Distancias de paro y de centrado
`AJUSTE_DER`	0	Corrige si un motor es más fuerte
`refs[]`	se mide	Colores de referencia

10. Glosario

Término	Significado
MCU	Microcontrolador: computadora pequeña y puntual para controlar hardware
MPU	Microprocesador: computadora grande que corre un sistema operativo (Linux)
RPC	Llamada a procedimiento remoto: un procesador le pide a otro que ejecute una función
I²C	Bus de 2 cables (SDA datos, SCL reloj) que comparten varios sensores
ToF	*Time of Flight*: mide distancia por el tiempo que tarda la luz en ir y volver
PWM	Prender y apagar muy rápido para regular la potencia; 0 = apagado, 255 = máximo
Puente H	Circuito que deja girar un motor en ambos sentidos
Lazo abierto / cerrado	Sin / con medición del resultado para corregir
Control P	Corrige en proporción al error
***Flood fill* (BFS)**	Búsqueda en anchura: calcula la distancia de cada celda a la meta
Encoder	Sensor que cuenta las vueltas de la rueda
IMU	Sensor de giro y aceleración
HSV	Forma de describir un color por tono, saturación y brillo
CSG	Operaciones booleanas entre sólidos 3D (unión, resta, intersección)